

Depth First Traversal

Ad esempio un Token

Con questa task vogliamo risolvere il problema in cui data una risorsa la quale deve essere condivisa in mutua esclusione con tutti i nodi del sistema

Più formalmente stiamo cercando un protocollo che

Partendo da un single initiator, permetta ad un token di raggiungere sequenzialmente ogni entità

Assunzioni e restrizioni

Stiamo studiando la task del Traversal in un sistema sotto le assunzioni di:

- IC $\leadsto$ connectivity, total reliability, bi directional links
- UI $\leadsto$ unique initiator

Stati possibili

Gli stati possibili in cui si trovano i nodi sono

- Init
- Idle
- Visited
- Done

Configurazione iniziale e finale

Analizzeremo ora le possibili configurazioni iniziali e finali:

- Configurazione iniziale

$$\exists! x \in E \text{ t.c. } status(x) = \text{Init} \text{ AND } \forall y \in E, y \neq x \text{ t.c. } status(y) = \text{Idle}$$

n possibili configurazioni iniziali

- Configurazione finale

$$\forall x \in E : status(x) = \text{Done} \leadsto \text{L'unica configurazione finale}$$

Idee dietro al protocollo

L'idea alla base è quella di inoltrare il Token quanto più possibile

Se il Token viene ricevuto da un nodo già stato visitato

il nodo rimanda il Token indietro e marca l'arco come back-edge

Messaggio di Back-edge

Successivamente se il Token non può più essere inoltrato allora viene inviato indietro

Messaggio di Reply

Sintanto che non si trova un nodo che presenta un'arco ancora mai attraversato dal Token

Implementazione ad alto livello del protocollo

Il nostro protocollo prevede 4 possibili azioni

Siano $x, u \in V$ i.e. $u \in N(x)$

1) Quando x riceve il Token per la r -esima volta:

- ricorda chi glielo ha consegnato $\leadsto$ Father
- manda il Token ad $u \in N(x)$ non visitato
- Aspetta il Reply di u

Assumendo che u non sia visitato

2) Quando u riceve il Token:

Se u è già stato visitato

rimanderà il Token al padre comunicandogli che l'arco è un back-edge

Altrimenti

senza il padre

lo manderà a tutti $N(u)$ prima di rimandarlo al padre

3) Quando x riceve Reply da u invia il Token ad un altro nodo ancora non visitato in $N(x)$

ovviamente meno che il padre

4) Quando x ha ricevuto Reply o Back-edge da tutti i vicini

Allora x manda Reply al suo Father

Msg Complexity

Sapendo che i tipi di msg possibili sono 3:

- Token
- Reply
- Back-edge

Sia $e = (x, y)$ un arco, i msg che possono attraversare e sono:

- 1) Il Token inviato da x verso y
- 2) Il msg di Reply o di Back-edge

Capiamo così che VecE possono al massimo 2 msg, portando quindi la msg complexity ad essere

$$Msg(Traversal) = 2m = \Theta(m)$$

Time Complexity

Sotto le Assunzioni di Sincronia e/o di bound dei ritardi analizziamo il tempo d'esecuzione

Per quanto descritto in precedenza il protocollo invia msg in modo strettamente sequenziale

Infatti, visto che ogni msg ci costa un'unità di tempo

$$Time(Traversal) = 2m = \Theta(m)$$

Ottimizzazione del protocollo

Per migliorare le performance di un protocollo si adottano solitamente 2 strategie:

- 1) Ridurre il numero di msg
- 2) Introdurre più concorrenza $\rightarrow$ nel senso di mandare più msg in parallelo

Nel nostro caso utilizzeremo la 2ª opzione

Nello specifico aumenteremo il numero complessivo di msg inviati in modo da eseguire un numero inferiore di operazioni sequenziali

Idea dietro Traversal

Per diminuire la sequenzialità del protocollo vogliamo inviare il Token solo ai nodi che non sono ancora stati visitati:

Introduciamo quindi:

- Due nuovi msg

Visited con cui un nodo notifica di aver ricevuto il Token

Ack con cui si risponde al msg Visited

- Un nuovo stato Available

in cui vengono posti i nodi in Idle che ricevono il msg Visited

Quindi le modifiche che applicheremo sono:

- 1) Quando il nodo x riceve il Token per la 1° volta manda parallelamente un msg Visited a tutti i suoi vicini e aspetta un msg di risposta Ack

- 2) Quando x invia il Token ai vicini

lo invia esclusivamente verso i vicini da cui non ha ricevuto Visited precedentemente

nuovo

oppure a cui non ha inviato Ack

Aumentando quindi il numero di msg trasmessi saremo sicuri che il Token verrà inviato solo ed unicamente ai nodi che non lo hanno mai ricevuto prima

Oss

Si può notare come tramite queste modifiche abbiamo eliminato l'utilità del msg Back edge

Analisi Time Complexity

Considerando i 4 tipi di msg

Token, Reply, Visited, Ack

Analizziamo il tempo complessivo studiando il numero di msg sequenziali:

- Token e Reply:

$\forall x \in V \setminus \{Init\}$ x invia un solo Token ed un solo Reply $2(n-1)$ unità di tempo

- Visited e Ack:

$\forall x \in V$ x invia parallelamente Visited a tutti i vicini e attende il msg di Ack
all'interno dell'unità di tempo successiva 2n unità di tempo

In totale quindi:

$$\text{Time}(\text{Traversal}) = 2(n-1) + 2n = 4n - 2 = \Theta(n)$$

Il che è un netto miglioramento rispetto a $\text{Time}(\text{Traversal}) = \mathcal{O}(m)$

Analisi Message Complexity

Contiamo ora il numero di msg inviati all'interno del sistema durante il protocollo

- Il numero di Token e Reply risulta essere $2(n-1)$ grazie all'analisi precedente
- Il numero di Visited invece sono in numero

$$|N(s)| + \sum_{x \in V \setminus \{s\}} |N(x)| - 1 = 2m - (n-1) \quad \text{con } s \text{ l'init}$$

Visto che s invia Visited ad ogni vicino e $\forall x \in V$ lo invia a tutti tranne che al padre

- Il numero di Ack è $2m - (n-1)$ visto che ad ogni Visited viene associato un Ack

In totale quindi:

$$\text{Msg}(\text{Traversal}) = 4m - 2(n-1) + 2(n-1) = 4m = \mathcal{O}(m)$$